



AULA 2

Os 4 Pilares da Orientação a Objetos



Objetivos de aprendizagem da aula

Ao final desta aula, você irá:

- Entender conceitos básicos de orientação a objetos (classes, objetos, variáveis e métodos);
- Compreender conceitos de encapsulamento, herança, polimorfismo e abstração;
- Reconhecer e entender o propósito de classes abstratas;
- Entender e diferenciar os conceitos de associação, agregação e composição.

Na aula anterior, estudamos alguns conceitos básicos da linguagem Python, que utiliza o paradigma da orientação a objetos (OO).

Nesta segunda aula, iremos nos aprofundar na programação orientada a objetos (POO). Primeiro, iremos entender o que é a POO e detalhar os quatro pilares da OO. Você vai compreender e saber para que servem o encapsulamento, a herança, o polimorfismo e a abstração, e como utilizá-los para construir códigos que sigam boas práticas. Você também vai entender a diferença entre associação, agregação e composição, e como usar esses conceitos em seus programas orientados a objetos.

Que história é essa?

Paradigmas de programação

A programação procedural, também chamada de estruturada, é baseada no conceito de chamadas a procedimentos (rotinas, métodos e funções). Já a programação orientada a objetos (POO) é uma aproximação da visão de mundo que temos, em que trabalhamos com transformação, manipulação e utilização de objetos.

Vamos entender melhor suas diferenças?

Na **programação procedural**, um programa é escrito como uma sequência de instruções ou função. Ele pode ser quebrado em subprogramas. Observe na imagem e após clique na seta

Interativo

Descrição do interativo

Classes e objetos

Em POO, as classes estão para objetos assim como formas estão para bolos. Por exemplo, podemos preparar vários bolos a partir de uma mesma forma: bolo de chocolate, laranja, fubá, etc. O mesmo acontece com os objetos.



Fique ligado!

Podemos criar vários objetos a partir de uma mesma classe, mas, assim como cada bolo tem um sabor, cada objeto terá sua própria identidade.

Clique nos itens a seguir e conheça mais detalhes que diferenciam as classes dos objetos:



Interativo

Descrição do interativo

É importante destacar que o comportamento de um objeto é definido por sua respectiva classe a partir de métodos, um conjunto de rotinas que operam sobre os dados.

Já o estado de um objeto é definido por sua respectiva classe a partir de variáveis de instância, que representam os dados do objeto.

Vamos imaginar um exemplo mais próximo do dia a dia?

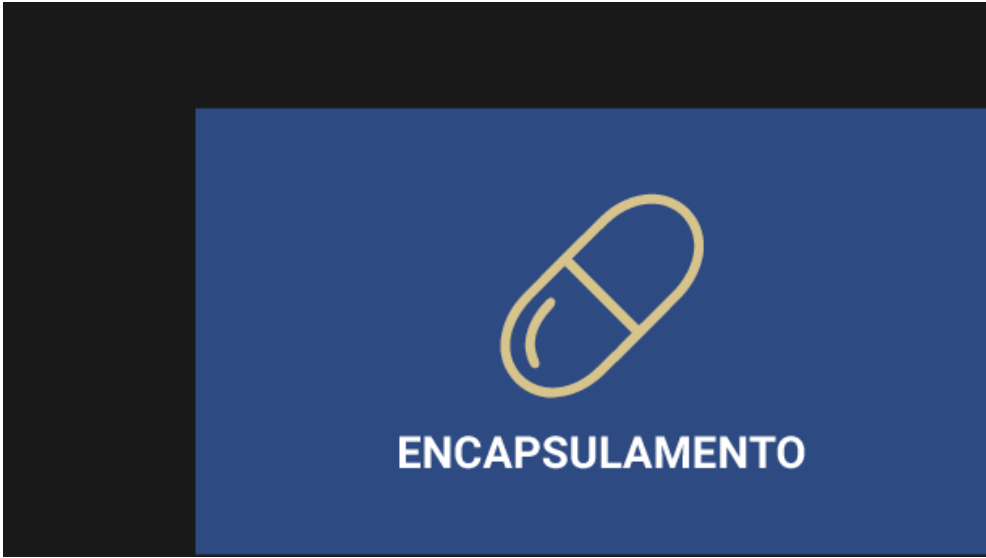


Interativo

Descrição do interativo

Como vantagens da orientação a objetos, é possível destacar que estes são representados de forma clara, com responsabilidades bem definidas e independentes de outros objetos. Isso permite um código mais claro e de fácil manutenção, além de possibilitar que trechos de códigos sejam facilmente reutilizados em outros sistemas.

Mas quais são os pilares da POO? Clique nos itens a seguir para conhecê-los.



Interativo

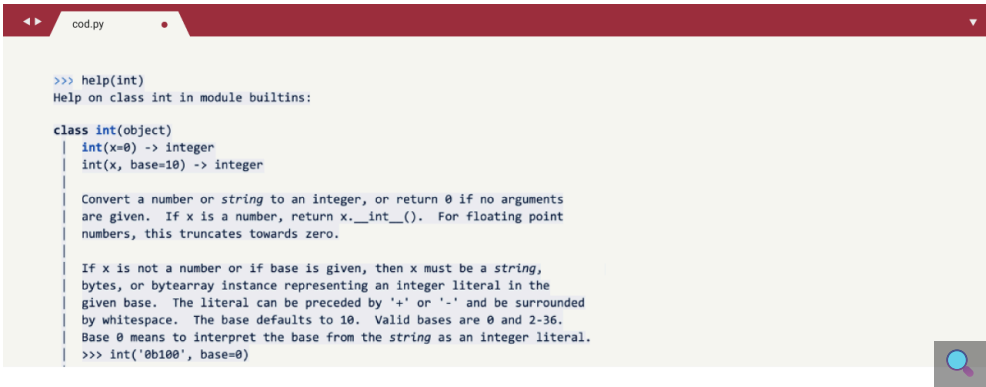
Descrição do interativo

Ainda nesta aula, você vai conhecer mais a fundo cada um desses pilares.

Como você viu, uma classe é uma estrutura de dados que une variáveis (propriedades) e funções (comportamentos).

Em Python, tudo deriva de uma classe, inclusive tipos básicos, como números (por exemplo, *int*) e textos (por exemplo, *str*). Veja o que acontece quando executamos o comando `help(int)`:

Observe que o tipo *int* deriva da classe *int*, que, por sua vez, herda da classe *object* os atributos e métodos comuns a todas as classes.



Nessa linguagem, as classes podem ser definidas em qualquer lugar, inclusive dentro de funções.

Usualmente, as classes são definidas r
ou em módulos próprios, para deixar o

A sintaxe para definir uma classe é:

cod.py

Interativo

Descrição do interativo

Métodos

Dentro das classes, é possível definir os métodos que representam as ações que os objetos criados a partir delas poderão realizar.

Um método recebe pelo menos um parâmetro: a instância da classe, representada pela palavra reservada *self*. Esse parâmetro permite saber a que instância se referem os atributos.

Na declaração do método, é preciso declarar *self* explicitamente, mas, ao chamar o método em um objeto, *self* é passado automaticamente.

Fique ligado!

O parâmetro *self* pode ter outro nome, mas *self* é uma convenção (padrão PEP8). Métodos podem receber qualquer quantidade de parâmetros, que podem ser obrigatórios ou não (declarados com um valor *default*).

Vamos a um exemplo?

Definimos uma classe *Pessoa*, que tem dois métodos:

- `__init__`, que inicializa o atributo nome durante a criação da instância; e
- `__str__`, converte uma *Pessoa* para *string*. Observação: quando passamos um objeto para o comando *print*, este vai buscar pelo método `__str__` para saber como representar o objeto na saída padrão.

```
cod.py

1 class Pessoa:
2     def __init__(self, nome):
3         self.nome = nome
4     def __str__(self):
5         return self.nome
6
7 pessoa_1 = Pessoa("João")
8 print(pessoa_1)
9
10 pessoa_2 = Pessoa("Maria")
11 print(pessoa_2)
```

João
Maria

As variáveis *pessoa_1* e *pessoa_2* referenciam instâncias (objetos) da classe *Pessoa*, com valores distintos para o atributo *nome*.

Para chamar um método em um objeto, basta escrever os nomes da variável que o referencia e do método separados por um ponto, passando os parâmetros cabíveis. No exemplo, estamos chamando os métodos *soma* e *media* no objeto referenciado por *minha_lista*.

```
cod.py

1 class ListaDeNumeros:
2     def __init__(self, numeros):
3         self.numeros = numeros
4     def __str__(self):
5         return ", ".join([str(n) for n in self.numeros])
6     def soma(self):
7         return sum(self.numeros)
8     def media(self):
9         return self.soma()/len(self.numeros)
10
11 minha_lista = ListaDeNumeros([1,3,5,7,9])
12 print(f"A soma de {minha_lista} é {minha_lista.soma()}")
13 print(f"A média de {minha_lista} é {minha_lista.media()}")
```

A soma de 1,3,5,7,9 é 25
A média de 1,3,5,7,9 é 5,0

Atributos

Existem três tipos de variáveis que podem ser declaradas dentro de uma classe:

- Atributos de classes**

▼
- Atributos de instância**

▼
- Variáveis de métodos**

▼

Clique nas setas para entender melhor.

Exemplo 1

Criamos a classe *Pessoa* com o atributo de classe *num_pessoas*, utilizado para contar quantas instâncias de *Pessoa* foram criadas. No método *__init__* criamos um atributo de instância *nome*.

Interativo

Descrição do interativo

Vamos entender melhor vendo alguns exemplos?

Introdução à orientação a objetos

Neste vídeo, você vai ver alguns exemplos comentados de código em Python, que ilustram os principais conceitos de orientação a objetos já estudados até aqui.



Agora que estamos um pouco mais familiarizados com classes e objetos, vamos detalhar cada um dos quatro pilares da orientação a objetos: **encapsulamento, herança, polimorfismo e abstração**.

Encapsulamento

Como vimos anteriormente, **o encapsulamento é um recurso da orientação a objetos que permite agrupar um código e os dados por ele manipulados em uma única entidade, protegendo-os de interferências externas, acesso indevido e/ou utilização/modificação inadequada** por outro trecho de código definido fora do código encapsulado.

Uma classe naturalmente encapsula o código restringir o acesso a um membro (atributo ou método) para a classe em questão (*private*) ou para quem a instancia.

Clique na parte azul ou na parte bege para visualizar o exemplo.



Interativo

Descrição do interativo

Em OO, é uma boa prática tornar os atributos privados, uma vez que é responsabilidade de cada classe controlar seus atributos.

Para isso, algumas linguagens utilizam o conceito de palavras-chave para definir o nível de visibilidade, e Python considera que existem os tipos *public* e *non-public*. Para marcar como *non-public*, usam-se dois *underscores* (__) antes do nome do atributo ou método.



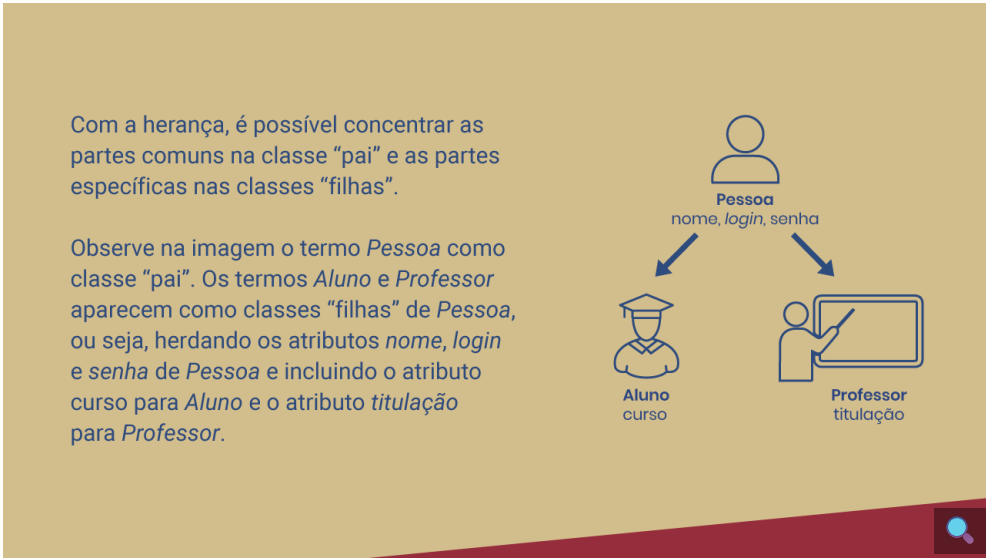
Não se usa o termo *private* para Python, pois nenhum atributo é de fato mantido como *private*. É apenas exibido um alerta de que você não deveria estar tentando acessá-lo. Apesar de ser possível acessar e até alterar o valor de um atributo *non-public* em Python, isso não é uma boa prática.

Mantendo o foco

Vale ressaltar que as mesmas regras de acesso para atributos valem para métodos. Como boa prática, os atributos de instância são *non-public* e a maioria dos métodos, *public*. Isso possibilita que a conversa entre objetos seja feita por troca de mensagens, ou seja, acessando seus métodos, em vez de um objeto manipular diretamente atributos que não sejam seus.

Herança

A herança permite que as características e os comportamentos descritos de uma classe “pai” (superclasse) sejam herdados por sua classe “filha” (subclasse).



Uma relação de herança significa que a subclasse herda os membros da superclasse (atributos e métodos).

Estas são as principais vantagens da utilização da herança:

- Evita código duplicado.
- Facilita a manutenção, pois há apenas um lugar para fazer alterações em caso de modificações (superclasse), sem precisar modificar as subclasses.
- Possibilita definir um protocolo comum para um grupo de classes relacionadas.
- Garante que todas as subclasses, tanto atuais como futuras, de uma superclasse tenham os mesmos métodos que serão herdados.
- Permite estabelecer um “contrato” mínimo do que as subclasses – em qualquer nível – poderão fazer.

Fique atento à utilização da herança:

✓

Use a herança quando...

...uma classe for o tipo mais específico de uma superclasse.
Por exemplo: *Morango* (subclasse) e *Fruta* (superclasse).

...tiver um comportamento compartilhado entre várias classes do mesmo tipo geral.
Por exemplo: *Pessoa* (superclasse), *Aluno* e *Professor* (subclasses).

✗

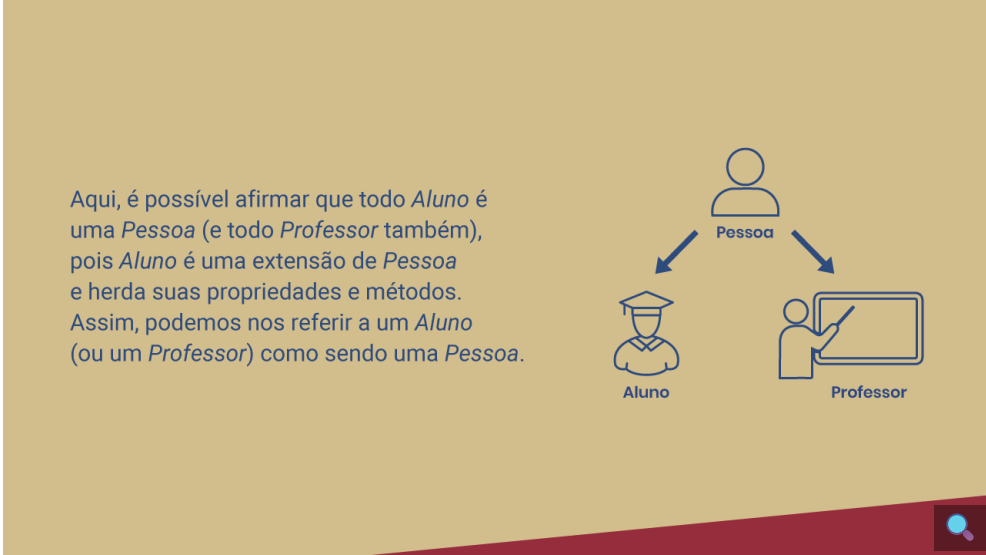
Não use a herança...

...apenas para poder reutilizar o código de outra classe, se o relacionamento entre a superclasse e a subclasse violar uma das regras anteriores.
Por exemplo: um código especial de impressão de *Torradeira* sendo utilizado em *Piano*. Nesse caso, é necessário mover o código para uma nova classe *Impressora*, que seria utilizada pelas classes *Torradeira* e *Piano*.

...se a subclasse e a superclasse não passarem no teste É-UM.
Por exemplo: *Suco* É-UMA *Bebida*, mas *Bebida* NÃO-É um *Suco*.

Polimorfismo

Para falar de polimorfismo, vamos retomar um exemplo:



Polimorfismo é a capacidade de um objeto poder ser referenciado de várias formas (o que não significa que o objeto pode se transformar em outro tipo).

Se tivermos um método que espera receber um objeto do tipo *Pessoa*, ele pode receber, em vez disso, um objeto do tipo *Aluno* ou do tipo *Professor*.

Mantendo o foco

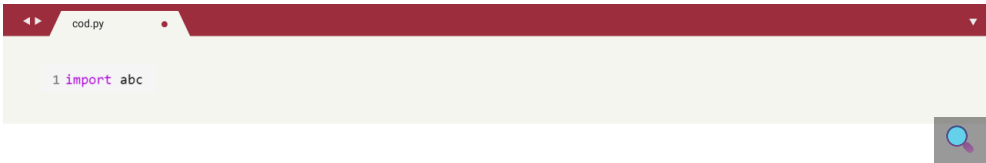
O polimorfismo é um recurso que permite projetar códigos com alto nível de abstração (pensando em superclasses), sem se preocupar com níveis mais baixos (subclasses), que poderão, inclusive, ser criados posteriormente. Isso permite a criação de um código flexível, simples e de fácil manutenção. Usando o polimorfismo, é possível diminuir o acoplamento entre as classes, evitando que modificações no código resultem em modificações em outros lugares também.

Abstração

Ainda analisando o exemplo anterior, repare que não faz sentido termos em nosso sistema uma classe *Pessoa*, pois os usuários serão de tipos específicos, como *Aluno*, *Professor*, *Coordenador*, etc., mas nunca tão genéricos como *Pessoa*.

Entretanto, se a classe *Pessoa* for removida do programa, perderemos duas importantes vantagens: o reuso de código entre as subclasses de *Pessoa* e a flexibilidade de termos argumentos polimórficos nos métodos. A solução, então, é tornar *Pessoa* uma classe abstrata, significando que ela nunca poderá ser instanciada.

Em Python, uma classe abstrata deve conter pelo menos um método abstrato, e podemos criar uma classe abstrata herdando da superclasse para classes abstratas *ABC* (*abstract base classes*), do módulo *abc*. Para tal, deveremos importar este módulo no nosso código, como no exemplo a seguir.



Mantendo o foco

Não é possível instanciar uma subclasse de uma classe abstrata sem implementar todos os seus métodos abstratos. Ao tornar um método abstrato, ele deverá ser obrigatoriamente implementado em todas as subclasses de uma classe abstrata.

Vamos pensar um pouco sobre esses pilares?

Reflita

Como você viu, Python é uma linguagem de programação interpretada.

Em sua visão, quais são as consequências práticas de não utilizarmos corretamente o **encapsulamento**, a **herança**, o **polimorfismo** e a **abstração** em nossos programas orientados a objetos?

Associação, agregação e composição

Os conceitos de associação, agregação e composição permitem o relacionamento entre objetos.

Para você entender melhor, vamos relembrar o exemplo da classe *Aluno*. Imagine que você queira ampliar essa classe para guardar seu professor orientador:

Nesse caso, seria possível criar nessa classe um atributo **nome_orientador** para guardar apenas o nome do professor orientador, e outros atributos para guardar as demais

Interativo

Descrição do interativo

Agora, imagine que o aluno tenha um histórico de ocorrências, como data de matrícula, notas obtidas, disciplinas cursadas e reprovações, entre outras. Nesse caso, temos um exemplo de **composição**.

Tanto a agregação quanto a composição são relacionamentos todo-
parte.



Interativo

Descrição do interativo

Vamos analisar alguns exemplos de códigos?

Os quatro pilares da orientação a objetos

Neste vídeo, vamos retomar os conceitos teóricos dos quatro pilares, mas examinando alguns exemplos de códigos.



Referências

Clique aqui para acessar as referências e créditos desta aula.

